Language and Compilers

Dr. Khaled Morsy

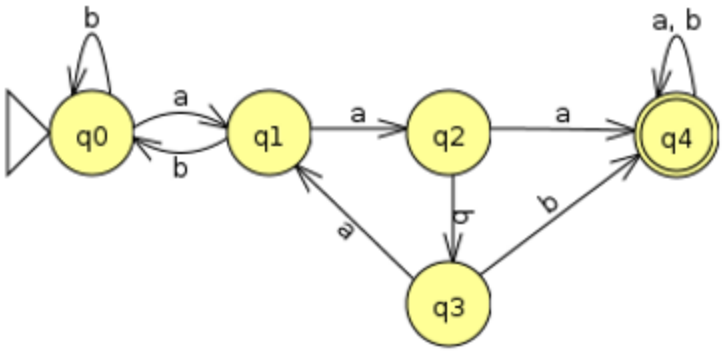
Sheet (3)

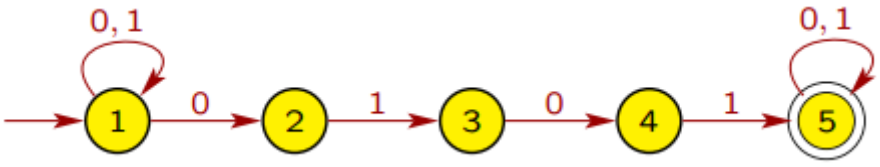
1- Consider the regular expression R = (a|ba)*baab, that matches {baab , abaab, aaaabaab, bababaab, ,……}
   (a) Draw the equivalent NFA. (Use the algorithm that based on concatenation, union and Kleene-star to build the NFA from regular expression).
   (b) Write down the algorithms of subset construction and ε-closure
   (c) Convert into DFA.

2- Give the regular expressions that represent the following strings over the alphabet {a,b,c}:
   - All strings containing even number of b's **solution**: [(a|c)* (bb)* (a|c)*]*
   - all strings containing exactly one a.
     **Solution** (b | c)* a (b | c)*

3- Show the Transition diagram for accepting unsigned numbers. (for example 234.5, 124E-2). Start state should be S0, and the other states are numbered S1, S2, and so on.

4- Given the following DFA states. Give a list of the accepted strings, then Show the transition table.




5- Draw a single NFA that recognizes the English words (the, then, they, thin, there, therefore, their)

(1) For the following grammar:

E→TE’

E’→+TE’ | ε

T→FT’

T’→*FT’ | ε

F→ a |b

1- Build the FIRST & FOLLOW sets
2- Construct the Parsing table

Our expression grammar from last time:

$E \rightarrow T\ E'$

$E' \rightarrow +\ T\ E'$

$E' \rightarrow \epsilon$

$T \rightarrow F\ T'$

$T' \rightarrow *\ F\ T'$

$T' \rightarrow \epsilon$

$F \rightarrow a$

$F \rightarrow b$

Its FIRST and FOLLOW sets:

| Symbol | FIRST | FOLLOW |
|---|---|---|
| E | {a, b} | {$} |
| E' | {ε, +} | {$} |
| T | {a, b} | {+, $} |
| T' | {ε, *} | {+, $} |
| F | {a, b} | {*, +, |

We use this information to construct the following parse table:

| | **A** | **b** | **+** | ***** | **$** |
|---|---|---|---|---|---|
| **E** | E → T E' | E → T E' | | | |
| **E'** | | | E' → + T E' | | E' → ε |
| **T** | T → F | T → F | | | |
| **T'** | | | T → ε | T' → * F T' | T → ε |
| **F** | F → a | F → b | | | |

Because no table entry contains more than one production, we know that the grammar is in LL(1) and that predictive parsing will successfully handle any input string.

---

(2) Use the resultant table and predictive parsing algorithm to parse (a+b*b)

(3) Give the **Bottom-up parsing** for the string a*b+a*b (the parse tree)

(4) Use shift-reduce algorithm to parse this string.